Build a Real-Time Chat App with React and Firebase

DST4010 FALL 2026

Group Project

Scenario

You have been asked to build a small messaging application for students. Users should be able to create an account, sign in, find another user, and exchange messages that appear without refreshing the page. React will power the interface; Firebase Authentication and Cloud Firestore will provide identity, data storage, and real-time updates.

Learning outcomes

- Create a React application with Vite and organize it into reusable components.
- Configure Firebase Authentication and manage sign-in state.
- Store user profiles, conversations, and messages in Cloud Firestore.
- Use a real-time listener to display new messages automatically.
- Handle validation, loading states, errors, and responsive layouts.
- Apply Firestore Security Rules and explain why UI checks alone are not security.

1. Requirements and tools

Required tools:

- Node.js LTS and npm.
- A code editor such as Visual Studio Code.
- A modern browser.
- A Google account for the Firebase Console.
- Git and a Git hosting account (recommended).

Functional requirements

Area	Acceptance requirement
Authentication	Register, sign in, sign out, and retain authentication state after refresh.
User directory	List registered users and select another user to start/open a conversation.
Messaging	Send and receive text messages; show sender, time, and message alignment.

Persistence	Messages remain available after reloading the application.
Real-time updates	A second signed-in user sees new messages without manually refreshing.
User experience	Responsive layout, input validation, loading indicators, empty states, and readable errors.
Security	Only conversation participants can read or send messages in that conversation.

2. Create the React project

Open a terminal and run the following commands:

```
npm create vite@latest react-firebase-chat -- --template react
cd react-firebase-chat
npm install
npm install firebase
npm run dev
```

Open the local URL printed by Vite. Confirm that the starter application loads before proceeding.

Checkpoint 1

- ☐ The Vite starter page loads in the browser.
- ☐ I can stop and restart the development server.
- ☐ I have initialized Git and made an initial commit (recommended).

3. Configure Firebase

1. Create a project in the Firebase Console: https://console.firebase.google.com/
2. Register a Web App and copy its web configuration.
3. In Authentication → Sign-in method, enable Email/Password.
4. Create a Cloud Firestore database.
5. Create a .env file in the project root and add the configuration values using the Vite VITE_ prefix.

```
VITE_FIREBASE_API_KEY=your-api-key
VITE_FIREBASE_AUTH_DOMAIN=your-project.firebaseapp.com
VITE_FIREBASE_PROJECT_ID=your-project-id
VITE_FIREBASE_STORAGE_BUCKET=your-project.appspot.com
VITE_FIREBASE_MESSAGING_SENDER_ID=your-sender-id
VITE_FIREBASE_APP_ID=your-app-id
```

Important: Do not commit your .env file. Add .env to .gitignore and provide a safe .env.example with placeholder values. Firebase web config is not a security boundary; access must be protected by Security Rules and Authentication. Never put service-account keys or private server credentials in a frontend app.

Create the Firebase module

Create src/firebase/config.js. Initialize Firebase using import.meta.env values, then export the app, auth, and Firestore instances. Restart Vite after changing environment variables.

```
import { initializeApp } from 'firebase/app';
import { getAuth } from 'firebase/auth';
import { getFirestore } from 'firebase/firestore';

const firebaseConfig = {
  apiKey: import.meta.env.VITE_FIREBASE_API_KEY,
  authDomain: import.meta.env.VITE_FIREBASE_AUTH_DOMAIN,
  projectId: import.meta.env.VITE_FIREBASE_PROJECT_ID,
  storageBucket: import.meta.env.VITE_FIREBASE_STORAGE_BUCKET,
```

```
  messagingSenderId: import.meta.env.VITE_FIREBASE_MESSAGING_SENDER_ID,
  appId: import.meta.env.VITE_FIREBASE_APP_ID,
};

const app = initializeApp(firebaseConfig);
export const auth = getAuth(app);
export const db = getFirestore(app);
```

Checkpoint 2

- ☐ Email/Password sign-in is enabled in Firebase.
- ☐ The application initializes without a configuration error.
- ☐ The .env file is excluded from version control.

4. Implement authentication

Build registration and login forms. Use Firebase Authentication methods such as createUserWithEmailAndPassword, signInWithEmailAndPassword, signOut, and onAuthStateChanged.

Tasks

- ☐ Create Register and Login components with controlled form inputs.
- ☐ Validate required fields and show helpful error messages.
- ☐ Create an authentication context or equivalent shared state.
- ☐ Subscribe to onAuthStateChanged and unsubscribe during cleanup.
- ☐ Show the chat page only when a user is authenticated.
- ☐ After registration, create a user profile document in the users collection with the user's UID as the document ID.

Checkpoint 3

- ☐ A new user can register and then sign in.
- ☐ Signing out returns the user to the login screen.
- ☐ Refreshing the page restores the signed-in session correctly.

5. Design the Firestore data model

Use the following suggested structure. You may propose an alternative, but document your design and implement security rules that match it.

User profile: users/{userId}

```
{
  "displayName": "Alex",
  "email": "alex@example.com",
  "photoURL": null,
  "createdAt": "server timestamp"
}
```

Conversation: conversations/{conversationId}

```
{
  "participants": ["userA", "userB"],
  "lastMessage": "See you tomorrow!",
  "updatedAt": "server timestamp"
}
```

Message: conversations/{conversationId}/messages/{messageId}

```
{
  "senderId": "userA",
  "text": "Hello!",
  "createdAt": "server timestamp"
}
```

Design notes:

- Use Firebase server timestamps for message creation and conversation updates.
- Use a consistent strategy to avoid duplicate one-to-one conversations.
- Query messages in chronological order and consider a sensible initial limit.
- Unsubscribe from real-time listeners when a conversation changes or a component unmounts.

6. Build the chat interface

Create a layout with a user/conversation sidebar and a message panel. On small screens, allow the user to move between the conversation list and active chat.

Suggested component structure

```
src/
  components/
    Navbar.jsx
    UserList.jsx
    UserSearch.jsx
    ChatWindow.jsx
    MessageList.jsx
    MessageInput.jsx
  pages/
    Login.jsx
    Register.jsx
    Chat.jsx
  context/
    AuthContext.jsx
  hooks/
    useMessages.js
  services/
    authService.js
    chatService.js
  firebase/
    config.js
```

Tasks

- ☐ Display a searchable list of users, excluding the current user.
- ☐ Select a user and open or create the corresponding conversation.
- ☐ Display sent and received messages differently.
- ☐ Send a message on button click and optionally on Enter (without breaking multiline input if used).
- ☐ Prevent empty or whitespace-only messages.
- ☐ Show a loading state while messages are fetched and an empty state when there are no messages.
- ☐ Scroll to the latest message when appropriate, without disrupting users reading older messages.

7. Add real-time message synchronization

Use Cloud Firestore's onSnapshot listener for the active conversation's messages. Keep message queries ordered by createdAt and handle the possibility of a message timestamp being temporarily unresolved.

Implementation guidance

- Subscribe to the selected conversation's messages when its ID changes.
- Update React state from snapshot results.
- Return the listener's unsubscribe function from the effect cleanup.
- Show an error if the listener fails, and avoid creating duplicate listeners.
- Use addDoc (or a deliberate ID strategy) to write messages with senderId, text, and serverTimestamp().

Checkpoint 4: two-user test

- ☐ Open the app in two separate browser sessions or profiles.

- ☐ Sign in as User A in one session and User B in the other.
- ☐ Send a message from A and confirm it appears for B without a manual refresh.
- ☐ Reply from B and confirm it appears for A.
- ☐ Reload both sessions and verify message persistence.

8. Secure the database

Implement Firestore Security Rules before considering the app complete. Rules must match your exact schema. At minimum:

- ☐ Require authentication for private data operations.
- ☐ Allow a user to read a conversation only if their UID is in participants.
- ☐ Allow message reads and creates only for conversation participants.
- ☐ Require senderId to match request.auth.uid on message creation.
- ☐ Validate message fields, types, and maximum text length.
- ☐ Prevent clients from arbitrarily changing conversation participants or impersonating another sender.
- ☐ Test rules using the Firebase Emulator Suite or Rules Playground where available.

Security warning: Do not use open rules such as allow read, write: if true in a deployed project. A UI check is not authorization; malicious clients can call Firebase directly. Rules should be tested against allowed and denied cases.

9. Testing matrix

Test	Expected result	Result
Register with valid details	Account is created and profile is stored	☐ Pass ☐ Fail
Register with invalid/duplicate email	Clear error; no broken UI	☐ Pass ☐ Fail
Sign in with wrong password	Access denied with a useful message	☐ Pass ☐ Fail
Send a valid message	Message is stored and appears in the chat	☐ Pass ☐ Fail
Send blank message	Message is not sent	☐ Pass ☐ Fail
Two users chat simultaneously	New messages synchronize in real time	☐ Pass ☐ Fail
Reload active conversation	Previously stored messages remain	☐ Pass ☐ Fail
Attempt unauthorized read/write	Security Rules deny the operation	☐ Pass ☐ Fail
Use narrow/mobile viewport	Layout remains usable without overflow	☐ Pass ☐ Fail
Disconnect or trigger a failure	Error/loading state is understandable	☐ Pass ☐ Fail